CSE 4113 – Internet Programming Lab

Project Report

Hopper

Team: Four-Eyed Raven

Submitted By

Team Member	Student ID
TODO: Add member 1 name	TODO: Add student ID
TODO: Add member 2 name	TODO: Add student ID
TODO: Add member 3 name	TODO: Add student ID
TODO: Add member 4 name	TODO: Add student ID

TODO: Add batch information

Department of Computer Science & Engineering

University of Dhaka

Submitted On

23.07.2026

Contents

Project Links

Item	URL
Public Git Repository	https://github.com/CREVIOS/Hopper
Deployed Application URL	https://hopper.farefin.com
API Docs (Swagger/Postman) – public link	https://hopper.farefin.com/api/docs
Demo Video	TODO: Add public demo video URL

1. Introduction

1.1 Problem Definition & Context

Hopper is a self-hosted compute provisioning platform intended for university environments. The repository describes it as a system that slices institution-owned bare-metal resources into isolated runtime environments that can be requested through a web interface, accessed over SSH, and governed through credit-based usage control and single sign-on integration. In the current implementation, the platform exposes these runtimes as Kubernetes-backed pod sessions while preserving the user-facing concept of provisioned virtual machine style workspaces.

The codebase shows that the project addresses several operational problems at once. First, shared academic compute infrastructure requires stronger isolation than ad hoc account sharing or manually provisioned hosts. Second, institutions need a practical way to manage fair usage, including per-session billing and automatic shutdown when credits are exhausted. Third, a university deployment must integrate with centralized identity management rather than maintaining a separate manual authentication system. Hopper therefore combines a browser-based control plane, an API layer, an orchestration service, a relational ledger-backed data model, and Keycloak-based authentication in one deployable platform.

1.2 Target Users & Use Cases

The verified user model in the repository includes at least three application roles: student, professor, and admin. These roles appear in the documented Keycloak realm configuration and are reflected in frontend routes and backend authorization logic. The system therefore serves both end users who consume compute sessions and privileged users who supervise access, credits, and platform status.

Students are the primary runtime users. Confirmed use cases include signing in through the authentication flow, viewing available plans, creating and terminating compute sessions, monitoring CPU and memory usage, opening browser-based development access, using SSH-related settings, and reviewing their credit balance and usage history. Professors and administrators have additional operational and oversight use cases, including allocating credits, inspecting node or platform statistics, reviewing users, and accessing administrative workflows exposed through dedicated backend and frontend routes.

1.3 Core Features

The repository confirms that Hopper provides a multi-service compute management workflow rather than a static informational website. Core platform features include authenticated session management through Keycloak, plan-based runtime creation, automated credit checks before provisioning, orchestration of pod lifecycle events through a Go service, and persistent tracking of sessions and ledger operations in PostgreSQL with

TimescaleDB support. These features are reinforced by the documented API routes, backend services, database migrations, and deployment manifests.

The frontend and backend code also confirm several user-facing capabilities beyond simple provisioning. The platform includes real-time metrics delivery through Server-Sent Events, browser-based terminal and file-management related functionality, administrative dashboards, SSH key management, notifications, and issue-reporting support. The infrastructure and documentation further show support for ingress-based web delivery, TLS termination, Keycloak-backed login and logout, NATS-based event propagation for billing and lifecycle messages, and Kubernetes deployment resources for a full cluster-based installation.

1.4 Tech Stack Overview

The frontend is implemented with SvelteKit 2, Svelte 5, TypeScript, and Vite. This choice is suitable because the project needs route-driven server rendering, interactive dashboards, and a structured component model for authenticated application pages. The dependency set and source tree also show use of lightweight Svelte stores and derived state rather than a heavyweight external state library. In practice, state management is handled through Svelte writable stores for concerns such as authentication, pod state, and notifications, together with route-level server load functions and local derived state in components.

The backend is implemented as a FastAPI application in Python 3.12, backed by Pydantic settings, SQLAlchemy, Alembic migrations, and supporting libraries for SSE, authentication, rate limiting, and asynchronous I/O. This stack is suitable because it supports typed API development, automatic OpenAPI documentation, asynchronous integration with external services, and relatively clear separation of routers, services, models, and middleware. The repository also contains a Go 1.23 orchestration service that uses gRPC and Kubernetes client libraries, which is appropriate for cluster-facing lifecycle management because the native Kubernetes tooling in Go is mature and directly aligned with the platform's orchestration requirements.

The primary database layer is PostgreSQL with TimescaleDB. This combination is suitable because Hopper needs conventional relational storage for users, sessions, roles, and accounting data, while also benefiting from time-series capabilities for metrics aggregation. Messaging and event distribution are handled through NATS JetStream, which fits the platform's need for lightweight asynchronous communication between the orchestrator and the API gateway, especially for billing and metrics events. Authentication is provided by Keycloak using OIDC flows, which is suitable for institutional single sign-on integration and role-based access control.

The deployment and hosting configuration is centered on Kubernetes, Nginx Ingress, and declarative infrastructure artifacts under `k8s/`, `charts/`, and `infrastructure/`. This is suitable because the platform provisions compute sessions as cluster workloads and de-

pends on ingress routing, service discovery, and namespace-level policy enforcement. The repository also confirms supporting infrastructure such as Docker and Docker Compose for local services, cert-manager for TLS certificate automation, and Cilium-enforced network policy definitions for zero-trust traffic control. Optional outbound email support is configured through Brevo or SMTP, but the exact production mail configuration remains environment-dependent.

2. System Architecture

2.1 High-Level Architecture Diagram

The deployed system is organized as a browser-facing frontend, a Python API gateway, a Go orchestration service, an identity provider, a relational database layer, and an event bus. The ingress configuration under `k8s/deploy/03-ingress.yaml` shows that external traffic is split across four path families: `/` for the frontend, `/api` for the gateway, Keycloak realm/resource paths for authentication, and a dedicated code-server proxy path for browser-based development access. The API gateway then coordinates synchronous database work, gRPC calls to the orchestrator, and asynchronous event handling over NATS.

At runtime, the orchestrator is the component that directly manages Kubernetes resources. The FastAPI gateway does not create pods itself; instead it creates or updates the application-facing database state, invokes the orchestrator over gRPC, and consumes follow-up events such as billing, pod lifecycle changes, and metrics updates. PostgreSQL stores the authoritative relational state for users, sessions, the credit ledger, notifications, queue entries, and other application records, while TimescaleDB features are used where available for metrics retention and hypertable support.

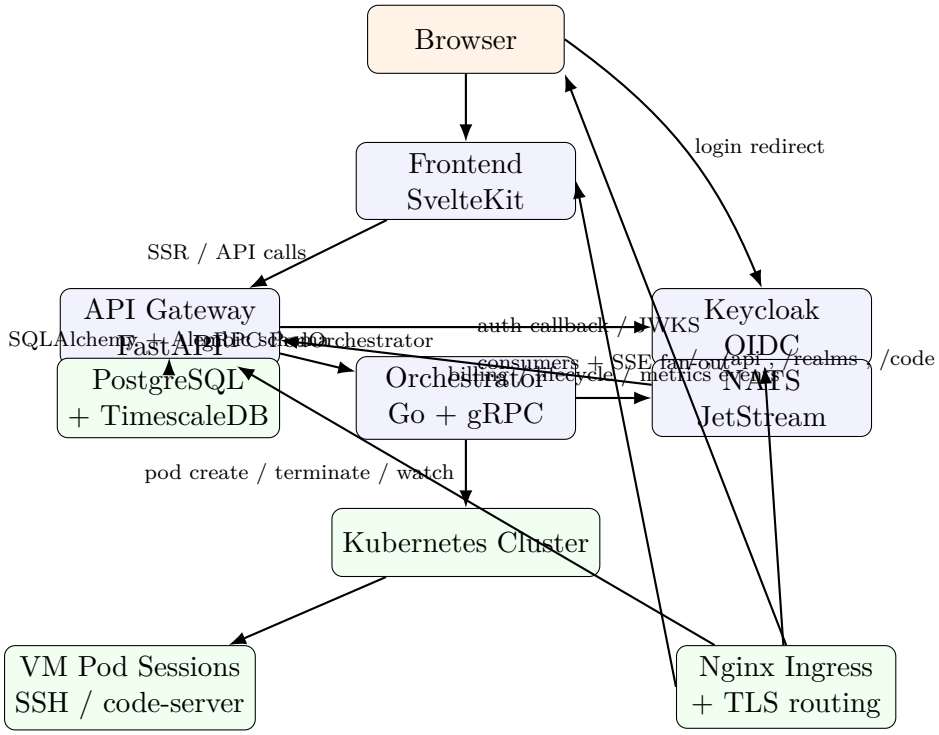

Figure 1: Verified high-level Hopper architecture derived from the frontend, gateway, orchestrator, and Kubernetes deployment files.

Figure 1 reflects the concrete implementation rather than an aspirational design. In particular, the current repository confirms the following communication paths: browser-to-frontend delivery over ingress, frontend-to-gateway HTTP calls, gateway-to-orchestrator gRPC calls, orchestrator-to-Kubernetes lifecycle control, orchestrator-to-NATS event publication, gateway-side NATS consumers for billing and metrics, and Keycloak-mediated authentication with cookie-backed application sessions.

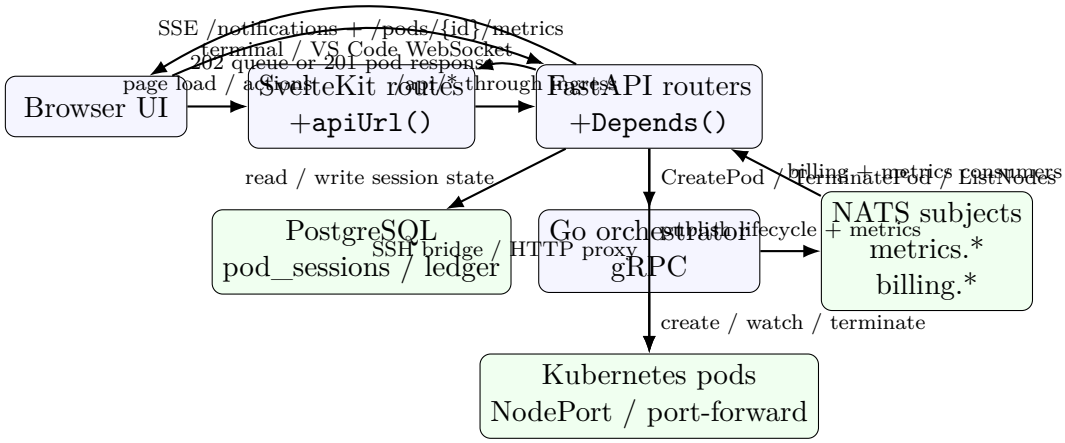

Figure 2: Main request and data flow for VM creation, monitoring, and interactive access.

The main operational flow shown in Figure 2 is also verified directly in code. The frontend issues requests through the `/api` path, the gateway checks identity and credits, the

scheduler either reserves capacity or enqueues the request, the orchestrator creates or terminates the Kubernetes workload, and asynchronous follow-up state reaches the UI through SSE streams, notification events, and interactive proxies.

2.2 Frontend Architecture

The frontend is a SvelteKit application organized around file-based routes in `frontend/src/routes`. Route-specific server load functions such as `+layout.server.ts`, `dashboard/+page.server.ts`, `admin/+page.server.ts`, and `pods/[id]/+page.server.ts` fetch authenticated data on the server side before rendering pages. The helper `frontend/src/lib/api/server.ts` resolves whether those server-side requests should go through an ingress-relative `/api` path or directly to an internal API base URL, depending on environment configuration.

Application-wide authentication state is coordinated in `+layout.server.ts` and `+layout.svelte`. The layout loader attempts `/auth/me` using the `session_token` cookie, falls back to `/auth/refresh` when necessary, and returns the authenticated user and balance to the shell layout. The authenticated shell then renders the sidebar, topbar, notification bell, and balance badge, and it also starts periodic refresh and balance polling timers in the browser. This indicates a mixed SSR-plus-client-refresh strategy rather than a purely client-side single-page architecture.

State management remains lightweight and is implemented with built-in Svelte mechanisms instead of an external library such as Redux or Zustand. The repository defines writable stores in `frontend/src/lib/stores/` for authentication, pod-related state, and notifications. Route components also use local derived state extensively, as seen in multiple `+page.svelte` files. This means the application follows a layered state strategy: server-loaded page data for initial render, writable stores for cross-component reactive state, and local derived values for view-level calculations.

The API layer is split between server-side and browser-side helpers. On the browser side, `frontend/src/lib/api/client.ts` wraps `fetch`, automatically includes credentials, retries once after a silent refresh on non-login `401` responses, and exposes a small SSE helper. On the server side, the route loaders call the backend directly through `apiUrl()`. This division is suitable for a SvelteKit application because it allows SSR data fetching, cookie forwarding, and browser reactivity without duplicating request logic in every route.

The real-time and interactive frontend features are implemented explicitly rather than implicitly. The notification store opens an EventSource connection to `/api/notifications/stream`, the pod detail page opens an EventSource to `/api/pods/{id}/metrics`, and the terminal component opens a WebSocket to `/api/pods/{id}/terminal`. Browser-based editor access is routed through the backend proxy path rather than by direct pod exposure. The repository therefore confirms three distinct frontend interaction modes: SSR page loading, credentialed REST-style API calls, and long-lived SSE or WebSocket channels.

2.3 Backend Architecture

The backend is implemented as a FastAPI-based API gateway in `services/api-gateway/app/main.py`. Its router registration shows a domain-oriented separation by capability: authentication, pods, credits, admin, settings, SSH keys, files, issues, notifications, and usage each live in their own router modules. Shared concerns are applied through dependency injection and middleware, including database sessions from `get_db()`, authentication from `get_current_user()`, audit logging, CORS control, and rate limiting. This structure is closer to a thin-router plus service-module design than to a heavy layered controller-service-repository stack.

The gateway is not a passive CRUD layer. It coordinates synchronous HTTP work with asynchronous background processes and remote orchestration. The `pods` router, for example, validates access, checks user credits, enforces per-user limits, invokes admission logic in `vm_scheduler` and `vm_queue`, persists `PodSession` state, and then calls the Go orchestrator through the async gRPC wrapper in `orchestrator_client.py`. If synchronous admission is not possible, the same router returns a queued response instead of failing immediately. This demonstrates an application-service style workflow centered in router handlers and helper service modules.

Several backend services are event-driven. During application startup, the gateway connects to NATS, starts the billing consumer, and starts the metrics consumer. The billing consumer processes `billing.deducted` and `billing.exhausted` events, applies idempotent credit deductions, manages grace-period behavior, and triggers user notifications. The metrics consumer stores periodic metrics snapshots in the database. The notification service persists rolling user notifications and also subscribes to pod lifecycle events so that `pod.started`, `pod.stopped`, and `pod.failed` can be reflected in the user interface. The result is a hybrid backend architecture combining request-response APIs, background event processing, and stateful session updates.

The admission queue is an especially important architectural feature. The service `vm_scheduler.py` elects a single leader through the `scheduler_leader` table and runs a first-come, first-served admission loop that considers cluster capacity computed from orchestrator node data plus current `PodSession` reservations. The queue service persists queued requests in `vm_queue_entries`. This means queueing is implemented inside the application database rather than delegated to an external job queue. The code explicitly documents that the loop is head-of-line and does not backfill smaller requests behind a blocked older request.

The Go orchestration service is separated into focused internal packages under `services/orchestrator/internal/`. The inspected file tree confirms packages for billing, gRPC endpoints, event publication, leader election, Kubernetes access, watchers, and pod management. The main process connects to NATS, creates Kubernetes clients, starts the gRPC server, and then runs singleton work that subscribes to events, reconciles watcher state, and publishes metrics. In other words, the orchestrator is the cluster-facing runtime

component, whereas the FastAPI application remains the externally facing control plane.

The current backend does not implement a dedicated repository abstraction layer. Database queries are performed directly with SQLAlchemy `AsyncSession` from routers and service modules. This is an important verified characteristic of the codebase and should not be misrepresented as a repository-pattern implementation. If a stricter persistence abstraction is desired later, that would be future work rather than a description of the current system.

2.4 Database Design

The database design combines normalized relational entities for identity, sessions, credits, and operational metadata with a time-series metrics table that can be promoted to a TimescaleDB hypertable when the extension is available. The foundational migration creates separate tables for users, accounts, transfers, ledger entries, pod sessions, and metrics. Later migrations adapt the original GPU-oriented session model into plan-based VM sessions, add notification and queueing support, and introduce API key and network-group support.

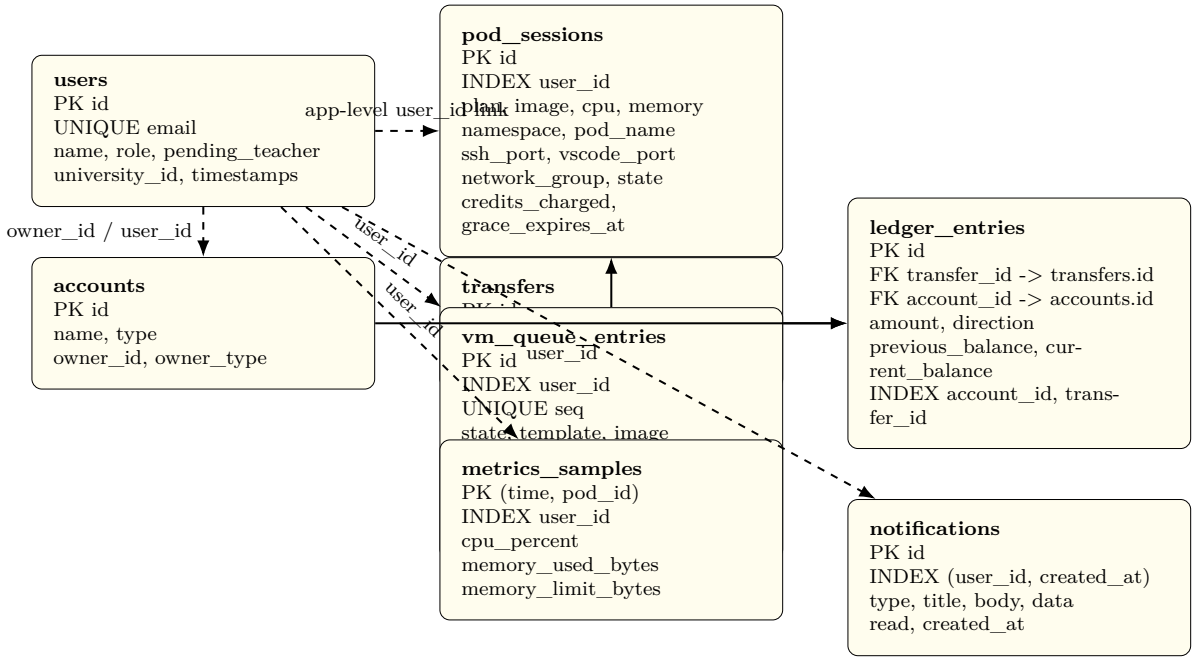

Figure 3: Core entity relationships derived from SQLAlchemy models and Alembic migrations. Solid arrows indicate database-enforced foreign keys; dashed arrows indicate application-level links stored by identifier only.

Figure 3 highlights an important characteristic of the current schema: not every logical relationship is enforced with a foreign key. The strongest database-enforced relationships are in the credit ledger, where `ledger_entries.transfer_id` references `transfers.id`, `ledger_entries.account_id` references `accounts.id`, and `vm_queue_entries.pod_session_id` references `pod_sessions.id`. By contrast, many user-owned records such as `pod_sessions`,

`notifications`, and `api_keys` store a `user_id` without a declared SQL foreign key. Ownership is therefore enforced primarily through application logic and authenticated route checks rather than through a dense relational constraint graph.

The credit subsystem is intentionally normalized into three tables: `accounts`, `transfers`, and `ledger_entries`. This supports double-entry accounting and avoids storing mutable balances as the only source of truth. The initial migration also creates indexes on `ledger_entries.account_id` and `ledger_entries.transfer_id` and uses PostgreSQL rules to prevent updates or deletes on ledger rows and transfers. That design decision is consistent with the service code, which treats the ledger as append-only and uses transaction-level locking to serialize deductions.

The VM lifecycle subsystem centers on `pod_sessions`. Each row stores the user-facing VM identity, selected plan, image, CPU and memory limits, namespace and pod name, exposed ports, current state, accumulated credits, and optional network-group data. Queueing is separated into `vm_queue_entries`, which freezes the requested plan and resolved image at enqueue time and adds a monotonically increasing `seq` field. The migration creating this table also defines a partial index on queued rows by sequence, which supports efficient first-come, first-served head-of-line admission scans.

The notification and metrics subsystems are also specialized rather than overloaded into a generic event table. Notifications form a rolling per-user window backed by a composite index on `(user_id, created_at)` because the main queries are recent notifications and unread counts. Metrics are stored in `metrics_samples` with a composite primary key `(time, pod_id)` and a separate index on `user_id`. The migration attempts to create a Timescale hypertable and retention policy when the database supports those functions, but it explicitly degrades gracefully to a regular PostgreSQL table when TimescaleDB is unavailable.

Several auxiliary tables exist outside the core ERD and contribute to operational completeness: `audit_logs`, `ssh_keys`, `issue_reports`, `email_codes`, `scheduler_leader`, and `api_keys`. These support auditing, SSH management, issue reporting, email verification and reset flows, leader election for the scheduler loop, and scoped programmatic access. Because Section 2 focuses on the main architectural data paths, they are described here textually rather than all being included in one overloaded ERD figure.

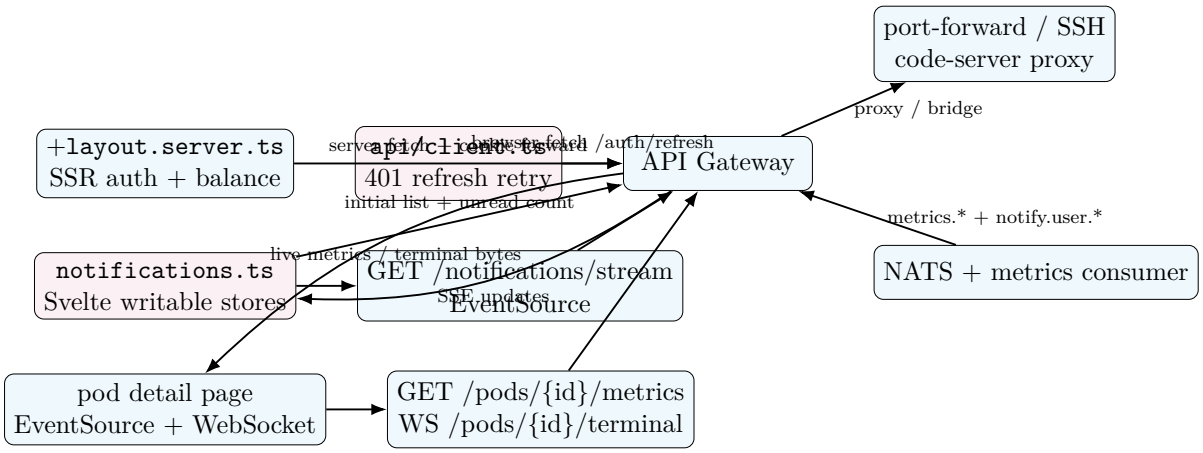

Figure 4: Verified automatic UI data-sync mechanisms: SSR refresh, SSE streams, and WebSocket proxying.

The repository confirms that automatic UI synchronization is implemented in multiple ways rather than through one generic push channel. As shown in Figure 4, the layout loader refreshes authentication state and forwards cookies server-side, the browser-side API client retries after silent refresh on eligible `401` responses, notifications are pushed over an `EventSource` stream, pod metrics are streamed through a separate `EventSource`, and terminal or browser-based editor access is proxied through WebSocket or HTTP upgrade paths in the gateway. This is a concrete, code-backed real-time strategy. It should therefore be documented as a combination of SSR data refresh, SSE fan-out, and backend-mediated interactive proxies rather than as a generic polling-only or WebSocket-only system.

3. API Documentation

3.1 API Design Overview

Hopper exposes a primarily REST-style HTTP API through the FastAPI gateway in `services/api-gateway/app/main.py`. The frontend API client hard-codes `/api` as its public base path, and the Kubernetes ingress rewrites that path before forwarding requests to the gateway service. Internally, the application routers are mounted directly at prefixes such as `/auth`, `/pods`, `/credits`, and `/admin`; externally, users reach them as `/api/auth`, `/api/pods`, `/api/credits`, and so on. The application metadata declares API version `0.1.0`, but the repository does not implement URI-based versioning such as `/v1`. The current versioning approach is therefore documentation-level rather than path-level.

The dominant request and response format is JSON over HTTPS. FastAPI and Pydantic models are used for request validation and typed response shaping, for example `LoginRequest`, `SignupRequest`, `CreatePodRequest`, `PodResponse`, `CreditBalanceResponse`, and `UserResponse`. The frontend client sets `Content-Type: application/json`

automatically for non-form submissions and includes browser credentials on every request. A smaller set of endpoints uses alternative transports where the feature requires them: file upload uses `multipart/form-data`, file download returns `application/octet-stream`, notification and metrics feeds use Server-Sent Events (SSE), and terminal or browser-editor sessions use proxied WebSocket or HTTP upgrade paths.

Authentication is enforced through the shared dependency `get_current_user()`. The main browser flow uses secure HttpOnly cookies, especially `session_token` and `refresh_token`, which are issued after successful Keycloak-backed sign-in. Programmatic access is also supported through Hopper API keys supplied either in `X-API-Key` or as `Authorization: Bearer hop_...`. The code explicitly restricts `read_only` API keys to `GET`, `HEAD`, and `OPTIONS` requests, while API-key creation and revocation require a real cookie-backed session. Authorization is role-aware rather than endpoint-global: the repository verifies at least three roles, namely `student`, `professor`, and `admin`.

Validation and error handling are a combination of schema validation and route-level checks. Pydantic enforces field structure, lengths, and constrained values, while the routers enforce application rules such as credit sufficiency, ownership, allowed domains, password policy, and role restrictions. The common error format is FastAPI's JSON `{"detail": "..."}` payload, although validation failures inherited from framework-level schema errors may return the standard structured `422` detail list. The codebase also shows consistent use of meaningful HTTP status codes such as `201 Created`, `202 Accepted`, `204 No Content`, `401 Unauthorized`, `402 Payment Required`, `403 Forbidden`, `404 Not Found`, `409 Conflict`, `422 Unprocessable Entity`, and `429 Too Many Requests`.

The API does not follow one universal pagination contract, but several repeatable conventions are implemented. Credit history and audit-log style listings use `limit` and `offset` query parameters. Notification and issue listing endpoints impose bounded maximums, and issue administration accepts a simple `status` filter. Usage endpoints accept a `range` parameter with confirmed values `1h`, `6h`, `24h`, and `7d` to control aggregation windows. These conventions are pragmatic and code-backed, but they are not wrapped in a separate global pagination abstraction at the current stage of the project.

3.2 Endpoint Reference

The report focuses on the important public application endpoints that are intended for user-facing or evaluator-facing interaction. Internal readiness and liveness handlers such as `/healthz` and `/readyz` are deliberately excluded from the main table because they are deployment-facing operational probes rather than end-user APIs.

Authentication and identity endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	`/api/auth/login`	Starts the OIDC login flow with PKCE, state, and nonce, then redirects the browser to Keycloak.	No	Public
POST	`/api/auth/signup`	Creates a local-account sign-up request with role selection and password-policy checks.	No	Public
POST	`/api/auth/verify-email`	Confirms an emailed verification code.	No	Public
POST	`/api/auth/resend-code`	Re-sends a verification or password-reset code.	No	Public
POST	`/api/auth/forgot-password`	Starts the password-reset flow for an existing account email.	No	Public
POST	`/api/auth/reset-password`	Resets a password after code validation and, where applicable, marks the email verified.	No	Public
POST	`/api/auth/login`	Performs direct email-password login and issues session cookies on success.	No	Public
GET	`/api/auth/callback`	Completes the OIDC authorization-code exchange after Keycloak redirects back.	No	Public callback endpoint
POST	`/api/auth/refresh`	Uses the refresh-token cookie to mint a new access cookie.	Refresh cookie	Session holder
GET	`/api/auth/me`	Returns the authenticated user's current profile and pending-teacher status.	Yes	Any authenticated user
POST	`/api/auth/logout`	Revokes the refresh token where possible, clears cookies, and redirects to Keycloak logout.	Yes	Any authenticated user
POST	`/api/auth/api-keys`	Creates an API key and returns the plaintext key exactly once.	Yes	Any authenticated user with session cookie
GET	`/api/auth/api-keys`	Lists the caller's API keys without revealing stored secrets.	Yes	Any authenticated user
DELETE	`/api/auth/api-keys/\{key\_id\}`	Revokes one of the caller's API keys.	Yes	Any authenticated user with session cookie

Pod lifecycle and interactive session endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	`/api/pods/plans`	Returns the plan catalogue exposed to the provisioning UI.	No	Public
GET	`/api/pods/`	Lists the current user's pod sessions.	Yes	Any authenticated user
POST	`/api/pods/`	Creates a pod synchronously when capacity is available, or returns `202 Accepted` with queue information when the request is enqueued.	Yes	Any authenticated user; `network_group` limited to professor/admin
GET	`/api/pods/availability`	Reports whether synchronous capacity is currently available together with queue length.	Yes	Any authenticated user
GET	`/api/pods/queue`	Lists the caller's active queue entries and queue positions.	Yes	Any authenticated user

Method	Route	Purpose	Auth	Authorized Role
DELETE	/api/pods/queue/\{entry_id\}	Cancels one of the caller's still-queued VM requests.	Yes	Queue-entry owner
GET	/api/pods/\{pod_id\}	Returns details of a specific pod session.	Yes	Pod owner
DELETE	/api/pods/\{pod_id\}	Terminates a pod session and releases its reserved resources.	Yes	Pod owner
GET	/api/pods/\{pod_id\}/metrics	Streams live pod metrics to the UI through Server-Sent Events.	Yes	Pod owner
HTTP proxy	/api/pods/\{pod_id\}/vscode/\{path\}	Proxies browser-based code-server traffic for an owned pod across GET, POST, PUT, PATCH, DELETE, and OPTIONS.	Yes	Pod owner
WS	/api/pods/\{pod_id\}/vscode/\{path\}	WebSocket upgrade path used by the proxied browser editor.	Yes	Pod owner
WS	/api/pods/\{pod_id\}/terminal	Provides interactive terminal access to a running pod.	Yes	Pod owner

Credits and usage endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	/api/credits/balance	Returns the caller's current credit balance and account identifier.	Yes	Any authenticated user
GET	/api/credits/history?limit=\dots\&offset=\dots	Returns the caller's credit ledger history using limit-offset pagination.	Yes	Any authenticated user
GET	/api/credits/teachers	Lists professor accounts with balances for the admin grant workflow.	Yes	Admin only
GET	/api/credits/students	Lists student accounts with balances for allocation workflows.	Yes	Admin or professor
POST	/api/credits/allocate	Grants or reallocates credits, with semantics that differ for admin and professor roles.	Yes	Admin or professor
GET	/api/usage/\{pod_id\}?range=1h\textbar6h\textbar24h\textbar7d	Returns historical usage series for a specific pod.	Yes	Any authenticated user
GET	/api/usage/summary/me/series?range=1h\textbar6h\textbar24h\textbar7d	Returns aggregated historical usage across all of the caller's pods.	Yes	Any authenticated user
GET	/api/usage/summary/me	Returns a recent usage summary for the caller.	Yes	Any authenticated user

Administration endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	/api/admin/users	Lists all users with role and teacher-approval metadata.	Yes	Admin only
GET	/api/admin/teacher-requests	Lists pending teacher sign-up requests.	Yes	Admin only
POST	/api/admin/teacher-requests/\{user_id\}/approve	Promotes a pending teacher request to the professor role.	Yes	Admin only
POST	/api/admin/teacher-requests/\{user_id\}/reject	Rejects a pending teacher request while keeping the user as student.	Yes	Admin only

Method	Route	Purpose	Auth	Authorized Role
GET	/api/admin/courses	Returns the current course list. The implementation is presently an empty proof-of-concept response.	Yes	Admin only
GET	/api/admin/nodes	Returns cluster node capacity and readiness data via the orchestrator.	Yes	Admin only
GET	/api/admin/stats	Returns aggregate dashboard statistics such as total users and active VMs.	Yes	Admin only
GET	/api/admin/active-vms	Lists all currently active VM sessions together with owner details.	Yes	Admin only
PATCH	/api/admin/users/\{user_id\}/role	Changes a user's role with safeguards against invalid or dangerous demotions.	Yes	Admin only
GET	/api/admin/audit-logs?limit=\dots\&offset=\dots	Returns recent audit-log entries using limit-offset pagination.	Yes	Admin only

Files, SSH keys, settings, notifications, and issue tracking

Method	Route	Purpose	Auth	Authorized Role
PUT	/api/settings/vscode	Accepts a VS Code settings blob. The current implementation acknowledges the request but still documents future persistence work in comments.	Yes	Any authenticated user
GET	/api/ssh-keys/	Lists the caller's SSH public keys.	Yes	Any authenticated user
POST	/api/ssh-keys/	Adds an SSH public key after format, duplicate, and per-user quota checks.	Yes	Any authenticated user
DELETE	/api/ssh-keys/\{key_id\}	Deletes one of the caller's SSH keys.	Yes	Key owner
GET	/api/files/\{pod_id\}/list	Lists files inside a running pod directory.	Yes	Pod owner
POST	/api/files/\{pod_id\}/upload	Uploads a file to a running pod using multipart form submission and SCP behind the gateway.	Yes	Pod owner
GET	/api/files/\{pod_id\}/download	Downloads a file from a running pod as an octet-stream attachment.	Yes	Pod owner
POST	/api/issues/	Creates a user issue report, optionally linked to a pod.	Yes	Any authenticated user
GET	/api/issues/me	Lists issue reports submitted by the current user.	Yes	Any authenticated user
GET	/api/issues/admin?status=\dots	Lists issue reports for administrative triage, optionally filtered by status.	Yes	Admin only
POST	/api/issues/admin/\{issue_id\}/resolve	Marks an issue report as resolved.	Yes	Admin only
GET	/api/notifications/	Returns recent notifications together with an unread count.	Yes	Any authenticated user
POST	/api/notifications/read-all	Marks all of the caller's notifications as read.	Yes	Any authenticated user

Method	Route	Purpose	Auth	Authorized Role
POST	`/api/notifications/\{notification\_id\}/read`	Marks one notification as read.	Yes	Notification owner
GET	`/api/notifications/stream`	Streams live notifications to the browser using Server-Sent Events.	Yes	Any authenticated user

3.3 Swagger / Postman Collection Link

The FastAPI gateway exposes the default Swagger UI, and the repository's ingress configuration makes it reachable publicly at https://hopper.farefin.com/api/docs. This URL is verified from the combination of the gateway's default FastAPI documentation settings and the ingress rule that forwards the external `/api` prefix to the backend service.

No Postman collection URL was found in the inspected repository. The report therefore retains that gap explicitly: TODO: Add a public Postman collection URL if the team publishes one later.

3.4 Sample Request & Response

Example 1: direct login request

The following example reflects the request body accepted by `LoginRequest` in `services/api-gateway/app/schemas/user.py`. On success, the handler also sets secure HttpOnly session cookies; those headers are omitted here for brevity and because tokens must not be reproduced in a report.

Listing 1: Representative login request to the Hopper API

```
curl -X POST "https://hopper.farefin.com/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "student@example.edu",
    "password": "[REDACTED]"
  }'
```

Listing 2: Representative login response body

```
{
  "id": "2f3ac4d8-7d3d-4f7b-a2df-0e4f07a8d421",
  "email": "student@example.edu",
  "name": "Example Student",
  "role": "student"
}
```

Example 2: pod creation request

This second example shows a provision request based on the verified `CreatePodRequest` and `PodResponse` schemas. The route may return `201 Created` for immediate admission or `202 Accepted` when the request is queued; the JSON below illustrates the immediate-creation path.

Listing 3: Representative pod creation request

```
curl -X POST "https://hopper.farefin.com/api/pods/" \
  -H "Content-Type: application/json" \
  -H "Cookie: session_token=[REDACTED]" \
  -d '{
    "plan": "small",
    "template": "ubuntu"
  }'
```

Listing 4: Representative pod creation response body

```
{
  "id": "9bc3d6ee-4b7a-4482-a9de-4948fdde2ce1",
  "user_id": "2f3ac4d8-7d3d-4f7b-a2df-0e4f07a8d421",
  "state": "running",
  "plan": "small",
  "image": "hopper/vm-ubuntu:22.04",
  "cpu": "1",
  "memory": "2Gi",
  "node_name": "worker-1",
  "namespace": "hopper-user-2f3ac4d8",
  "ssh_port": 32022,
  "vscode_port": 32080,
  "ssh_password": "[REDACTED]",
  "network_group": null,
  "created_at": "2026-07-19T12:00:00Z",
  "updated_at": "2026-07-19T12:00:30Z"
}
```

4. Authentication & Security

This section documents security mechanisms verified in the Hopper repository as of the inspected tree. It does not assert that the platform is free of vulnerabilities; gaps called out below were confirmed by reading application code, Kubernetes manifests, CI workflows, and automated tests.

4.1 Authentication Strategy

Hopper delegates identity to **Keycloak** (OpenID Connect). The FastAPI gateway in `services/api-gateway` is the application entry point: it exchanges credentials with

Keycloak, validates Keycloak-issued JWT access tokens, and exposes HTTP APIs to the SvelteKit frontend.

Dual sign-in paths. The repository implements two complementary browser flows:

- **OIDC redirect (SSO).** `GET/api/auth/login` generates PKCE (`S256`), `state`, and `nonce`, stores the PKCE verifier and state in short-lived HttpOnly cookies, and redirects the browser to Keycloak. `GET/api/auth/callback` verifies `state`, completes the authorization-code exchange with the PKCE verifier, validates the access token, upserts a local `users` row, and sets session cookies before redirecting to the dashboard.
- **Direct email/password login.** `POST/api/auth/login` uses Keycloak's resource-owner password grant against the `hopper-api` client (documented as requiring Direct Access Grants). On success it issues the same cookie set as the callback path. This supports the themed in-app login form without redirecting to Keycloak's hosted page.

Token and session lifecycle. Successful authentication sets HttpOnly, `Secure`, `SameSite=Lax` cookies:

- `session_token` — Keycloak access token (JWT); cookie `max-age` tracks Keycloak's `expires_in` (typically on the order of minutes).
- `refresh_token` — used by `POST/api/auth/refresh` to mint a new access token; refresh cookie lifetime tracks Keycloak's refresh TTL.
- `id_token` — retained for RP-initiated logout (`id_token_hint`).

One-time OAuth helper cookies (`oauth_state`, `oauth_pkce`) are cleared after callback. `POST/api/auth/refresh` reads the refresh cookie, calls Keycloak's token endpoint, rotates cookies on success, and clears all session cookies on failure. `POST/api/auth/logout` best-effort revokes the refresh token at Keycloak, clears local cookies, and redirects to Keycloak's end-session endpoint.

JWT validation. `app/middleware/auth.py` validates every access token with JWKS from Keycloak (`/protocol/openid-connect/certs`), checking signature, `exp`, issuer (external Keycloak URL), and audience (`hopper-api`). Realm roles are mapped to application roles `admin`, `professor`, or `student` (defaulting to `student` if no app role is present). JWKS is cached with periodic refresh and a forced refresh on unknown `kid` (key rotation).

Self-service account lifecycle. Signup (`POST/api/auth/signup`) creates users in Keycloak via the admin service account, stores a local row (teacher signups set `pending_teacher`), and emails a numeric verification code—*without* issuing a session until the mailbox is verified when `HOPPER_REQUIRE_EMAIL_VERIFIED` is enabled (default `true`). Verification codes and password-reset codes are stored only as SHA-256 hashes with TTL and attempt limits (`app/services/verification.py`). Password rules are mirrored in `app/services/password_policy.py` before Keycloak enforces the realm policy.

Programmatic access. Scoped API keys (`hop_` prefix) authenticate via `X-API-Key` or `Authorization:Bearerhop_...`. Only a hash of each key is stored; plaintext is returned once at creation. Key management endpoints require a real cookie session, not an API key.

Frontend session handling. Server-side rendering in `frontend/src/routes/+layout.server.ts` loads the user via `/auth/me`, forwards `Set-Cookie` headers from `/auth/refresh` when the access token is stale, and exposes `isAuthenticated` to child routes. The browser client (`frontend/src/lib/api/client.ts`) sends credentialed requests, retries once after silent refresh on eligible `401` responses, and redirects to `/login?session_expired=1` when refresh fails. E2E tests (`tests/e2e/specs/auth-and-access.spec.ts`) assert HttpOnly session cookies and cross-tenant API denial.

Local development exception. `frontend/src/routes/dev-login/+server.ts` performs a password grant and sets cookies only when SvelteKit `dev` mode is active; production builds return `404`. Default usernames in that handler are overridable via private environment variables and are intended for local workflows, not production.

4.2 Authorization & Role Management

Role model. Three application roles are defined consistently in Keycloak realm configuration, JWT parsing, and the `users` table: `student`, `professor`, and `admin`. Keycloak remains canonical for role assignment; admin actions call `app/services/keycloak_admin.py` to promote users (for example, approving a pending teacher to `professor`).

Backend enforcement. Protected API routes depend on `get_current_user()` (`app/dependencies.py`), which resolves either a verified session JWT or a valid API key to a `TokenPayload`. Representative checks verified in routers include:

- **Admin-only:** `/api/admin/*` via `_require_admin` (professors are explicitly rejected).
- **Professor or admin:** credit listing/allocation endpoints in `app/routers/credits.py`; `network_group` on pod create is limited to professor/admin in `app/routers/pods.py`.
- **Resource ownership:** pod, file, terminal, and VS Code proxy routes compare `PodSession.user_id` to the authenticated subject; queue entries and SSH keys are scoped the same way.
- **API key scope:** `read_only` keys are rejected for non-safe HTTP methods; `full_access` keys inherit the owner's role but cannot manage API keys.

Frontend route guards. Authenticated pages redirect anonymous users to `/login` from `+page.server.ts` loaders (dashboard, pods, credits, settings, and others). `admin/+page.server.ts` requires `user.role==='admin'`; `teacher/+page.server.ts` requires `professor`. These guards improve UX but are not a substitute for backend checks; the API enforces authorization independently (confirmed by E2E cross-tenant pod access

tests returning 403 or 404).

Teacher approval workflow. Users who sign up as "teacher" are stored as students with pending_teacher=true until an admin approves or rejects the request via /api/admin/teacher-requests/*. Role changes that would remove the last admin are blocked in the admin router.

4.3 Security Measures

Transport and edge hardening. Production ingress (k8s/deploy/03-ingress.yaml) enforces TLS redirect and sets HSTS, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy on public paths. The API gateway adds X-Content-Type-Options and Referrer-Policy on all responses (app/main.py). The SvelteKit server hook (frontend/src/hooks.server.ts) adds nosniff, DENY framing, and referrer policy on SSR responses where ingress snippets are unavailable.

CORS. CORSMiddleware allows an explicit origin list from HOPPER_CORS_ORIGINS with credentials enabled. Startup fails if * appears in the list alongside credentialed cookies. Production manifests set a single origin (https://hopper.farefin.com). /auth/refresh additionally rejects requests whose Origin header is not in the allowlist.

Rate limiting and abuse controls. SlowAPI decorators cap sensitive unauthenticated routes (login, signup, verification, API-key creation). After authentication, get_current_user applies per-user (user:<sub>) or per-key (apikey:<id>) moving-window limits via in-memory storage (app/core/limiter.py). Failed password attempts are counted per (client IP, email) tuple before returning 429. Pod creation has a separate per-user limit (HOPPER_RATE_LIMIT_POD_CREATE).

Input validation and sanitization. Request bodies use Pydantic models with length bounds (app/schemas/user.py, pod/credit schemas). Email domains can be restricted via HOPPER_ALLOWED_EMAIL_DOMAINS with exact suffix matching (not substring). File paths reject null bytes and newlines; remote paths are passed through shlex.quote before SSH (app/routers/files.py). SSH public keys are validated by prefix and fingerprint; per-user key count is capped with a transactional advisory lock.

Audit and error handling. AuditMiddleware logs mutating HTTP methods to audit_logs using user IDs derived from *verified* JWTs only. API errors generally return FastAPI JSON {"detail":"..."} with conventional status codes (401, 403, 429, etc.). Forgot-password and resend-code endpoints return uniform success responses to reduce email enumeration.

Secrets and configuration. Runtime settings load from environment variables prefixed with HOPPER_ (app/config.py, services/api-gateway/.env.example). Sensitive values intended for production—database URL, Keycloak admin client secret, Brevo API key, SMTP credentials—are documented as Kubernetes Secret references in k8s/deploy/02-apps.yaml rather than plain manifest env literals. GitHub Actions deploy and

staging jobs consume repository secrets (`.github/workflows/publish.yml`, `staging-validation.yml`) for SSH, tokens, and staging URLs; workflow files reference secret *names* only, not values.

Repository inspection for hardcoded credentials. Application Python/TypeScript sources do not embed production API keys or live passwords. `app/config.py` and `k8s/deploy/00-secrets.yaml` contain *documented local-dev defaults* (for example `hopper_dev` database credentials and `REPLACE_ME` placeholders) that must be rotated before non-dev deployment; comments in `00-secrets.yaml` warn against applying dev secrets over production. E2E bootstrap scripts read test passwords from environment variables with test-only fallbacks (`scripts/test/bootstrap_keycloak.py`). Load-test scripts expect tokens via environment variables, not committed secrets.

File upload and interactive access. Uploads require authentication, pod ownership, and a running pod; content is streamed to a temporary file and copied via SCP over an SSH port-forward. The repository does *not* define a maximum upload size or content-type allowlist on the gateway—a confirmed gap (see Section 4.4). Terminal and VS Code proxies re-verify JWT cookies (and CORS origin on WebSockets) at the gateway before forwarding to user pods.

Network isolation (deployment layer). Multi-tenant `NetworkPolicy` and `CiliumNetworkPolicy` manifests under `k8s/deploy/04-network-policies.yaml` and `k8s/network-policies/` restrict pod-to-pod and egress traffic; infrastructure security tests in `tests/security/` document how to run authorization and isolation probes against a staging cluster.

4.4 Known Vulnerabilities & Mitigations

The table below separates protections already present in code or manifests from limitations observed during inspection and recommended follow-ups. None of these rows imply complete elimination of the listed risk class.

Risk	Existing Protection	Remaining Limitation	Recommended Mitigation
Stolen session or refresh token	Short-lived access cookies; HttpOnly+`Secure` flags; refresh origin check; RP-initiated logout with revoke	No server-side session revocation list beyond Keycloak refresh revoke; XSS in frontend could still exfiltrate cookies if `Secure` were disabled	Keep access TTLs short; monitor Keycloak sessions; consider tightening CSP; continue HttpOnly-only token storage (verified in E2E)
Credential stuffing / login brute force	Per-route SlowAPI caps; failed-attempt window per IP+email; generic `401` message	In-memory buckets scale with worker×replica count and reset on pod restart	Move rate-limit state to Redis or ingress-level throttling for consistent cluster-wide ceilings
Cross-user data access (pods, files, metrics)	`get_current_user` on routes; `user_id` ownership checks; E2E cross-tenant denial tests	Frontend role checks alone would be bypassable without API enforcement (API is enforced, but UI must stay aligned)	Keep adding integration tests for new resources; treat SSR guards as UX-only

Risk	Existing Protection	Remaining Limitation	Recommended Mitigation
API key leakage	Stored as SHA-256 only; read-only scope; separate rate limit; keys cannot create keys	`full_access` keys equal session power for automation; plaintext shown once at create	Prefer read-only keys where possible; rotate/revoke on suspicion; audit `api_key.*` actions
Weak or reused passwords	Keycloak realm policy + gateway mirror; history enforced in Keycloak on reset	Password grant and Direct Access Grants increase exposure if client misconfigured	Prefer authorization-code+PKCE for human login where feasible; disable password grant in production clients if SSO-only
Email / account enumeration	Forgot-password and resend-code always return `200`	Signup returns `409` on duplicate email; login returns distinct `403` for unverified email	Accept trade-off or unify error copy further; ensure production sets domain allowlists
CSRF on cookie-authenticated POST	`SameSite=Lax`; OIDC `state`; refresh `Origin` allowlist	No double-submit CSRF token on all mutating forms; `Lax` cookies still sent on top-level cross-site GET navigations	Add CSRF tokens for sensitive POSTs if third-party integrations expand
Large or malicious file upload	AuthZ on pod owner; path sanitization	No max upload size; full file buffered to temp storage; filename not sanitized for path traversal on remote join	Enforce size limits at gateway/ingress; validate filenames; stream with backpressure
MITM on gateway→VM SSH	Access limited to authenticated owners	`StrictHostKeyChecking=no` for automated SCP/SSH from gateway	Use known-hosts pinning or in-cluster exec without password-based SSH where possible
Default or committed secrets	K8s Secret refs for prod DB/SMTP/Brevo/Keycloak admin; `.env.example` documents vars without live values	`00-secrets.yaml` ships dev defaults in git; `HOPPER_CODE_URL_SIGNING_SECRET` default in code; setting is unused in routers inspected; `HOPPER_DEBUG=true` in `02-apps.yaml`	Provision secrets out-of-band for prod; wire signing secret via Secret if feature is enabled; set `HOPPER_DEBUG=false` in production
Public API surface disclosure	Auth on sensitive routes	Swagger UI at `/api/docs` is publicly reachable per ingress (Section 3)	Restrict docs to authenticated admins or internal networks if required by policy
Multi-tenant network escape	Cilium/NetworkPolicy manifests; optional `tests/security/` suite	Policies require correct cluster CNI deployment; not re-verified in this report compile	Run `tests/security/run-security.sh` on staging; keep network policies in sync with orchestrator labels

Automated security test coverage. Unit tests cover password-policy errors, API-key dependency behavior, rate-limiter keying, and pod authorization edge cases (`tests/unit/`). E2E specs cover login cookies, role-scoped navigation, silent refresh, and anonymous `401` responses. Broader infrastructure isolation tests are documented but require a dedicated staging cluster and tokens supplied via environment variables (`tests/security/README.md`).

5. Testing & Quality Assurance

5.1 Testing Strategy

Hopper adopts a layered testing pyramid that keeps the majority of checks fast and GPU-independent. The strategy is captured in `docs/TESTING_PLAN.md` and operationalised through the CI pipeline in `.github/workflows/ci.yml`.

Unit tests (Python/pytest) target the API Gateway in isolation. All 411 cases run without any live infrastructure; every service, router, schema, and middleware module has dedicated coverage files under `tests/unit/` (47 Python modules). Key focus areas include the double-entry credit ledger, billing consumer state machine, password-policy enforcement, RBAC dependency checks, and session management.

Unit tests (Go/testing) exercise the orchestrator's internal packages and billing-contract layer using fake Kubernetes client-go sets and in-process stubs. Sixteen tests pass in `go-test.json` (internal K8s package), with a further 27 contract-level passes recorded in `go-contracts.json` (billing pricing, pod lifecycle state transitions, and resource-limit enforcement).

Integration tests (Python/pytest) in `tests/integration/` cover 18 modules against live Docker-based PostgreSQL, NATS JetStream, and Keycloak services spun up by the `python-integration` CI job. Modules include `test_credit_ledger_invariants`, `test_nats_event_flow`, `test_keycloak_integration`, `test_pods_api`, `test_auth_api`, and 13 others.

Frontend unit tests (Vitest) in `frontend/src/` verify 78 test cases across route server loads, API client behaviour, and Svelte component logic (including auth redirect rules, session refresh paths, and landing-page server guards).

End-to-end tests (Playwright) in `tests/e2e/specs/` span 6 specification files and 35 test cases as documented in `tests/e2e/COVERAGE_MATRIX.md`. Tests run against a mock-backed stack (SvelteKit dev server + Python mock API server) under the `e2e-mock` CI job. The final mock-suite run reported **27 passed (25.9s)**.

Load tests (k6) in `tests/load/` define five scenarios: class-start concurrent pod creation (30 VUs), steady-state metrics polling (100 VUs), spike to 150 VUs, class-end mass termination, and billing stress. Results are stored in `test-results/load/`. The scenarios run recorded 81 456 passing spike-check iterations; pod creation completed under 2 s on all sampled requests.

Helm chart validation runs on every push: `helm lint` and `helm template` are executed against `dev`, `staging`, and `prod` value overlays. Infrastructure chaos and security manifests reside under `tests/chaos/` and `tests/security/` for manual staging runs.

5.2 Test Coverage Summary

Coverage is measured independently per layer and aggregated by the `coverage-report` CI job. Figures are sourced from the artifacts recorded on **2026-07-18**.

Table 7: Automated test results and coverage by layer

Layer	Tests	Coverage	Artifact
API Gateway — Python unit (pytest)	411	81.68 %	`coverage/python/unit.xml`
API Gateway — Python integration (pytest)	18 modules	—	`tests/integration/`
Frontend — SvelteKit (Vitest)	78	66.08 %	`coverage/frontend/coverage-summary.json`
Orchestrator — Go internal (go test)	16	71.26 %	`coverage/orchestrator/orchestrator.out`
Orchestrator — Go contracts (go test)	27	—	`test-results/contracts/go-contracts.json`
E2E — Playwright mock suite	27 passed	—	`test-results/e2e/playwright-junit.xml`
Load — k6 spike scenario	81 456 checks	—	`test-results/load/scenarios-summary.json`

The lowest-covered API Gateway files are `routers/terminal.py` (41.90 %) and `routers/pods.py` (58.80 %). On the frontend, four files have 0 % coverage: `stores/notifications.ts`, `routes/dev-login/+server.ts`, `lib/confirm.svelte.ts`, and `routes/pods/queue/+page.server.ts`. In the orchestrator, `internal/k8s/pod.go` is the lowest at 57.80 %; the network-policy module (`netpol.go`) is at 100 %.

Confirmed testing gaps. The following areas currently lack automated coverage:

- Integration coverage XML (`coverage/python/integration.xml`) was not generated in the recorded run; Python integration tests exercise live services but no statement-level coverage artifact was produced.
- E2E test cases `TC-CREDIT-003` (grace-period auto-termination), `TC-ADMIN-003` (course creation UI), `TC-EDGE-003` (session-duration expiry), `TC-TMPL-002` (course template management), `TC-SCAV-001`, and `TC-SCAV-002` (scavenger plans) are marked *pending* or *future* in `tests/e2e/COVERAGE_MATRIX.md`.
- Cross-browser E2E is not enforced by default; Firefox and WebKit configurations exist but are opt-in via `E2E_CROSS_BROWSER`.
- Chaos experiments in `tests/chaos/` and security manifests in `tests/security/` require a live staging cluster and are excluded from the automated CI pipeline.

5.3 Bug Tracking & Resolution Log

All defects discovered during testing were tracked in `BUG_TRACKING_LOG.md`. The log covers the period 2026-07-17 to 2026-07-18 and records **20 bugs**, all with status *Resolved*. A representative sample is shown in Table 8.

Table 8: Representative bugs resolved during the testing period

Bug ID	Component	Root cause	Fix
BUG-2026-07-17-01	Frontend tabs	`Tabs.Trigger` swallowed extra props so per-page `onclick` handlers never fired.	Forwarded `...restProps` in `tabs-trigger.svelte`; added explicit state updates on admin and pod detail triggers.
BUG-2026-07-17-04	E2E pod lifecycle spec	Tests assumed conditions stricter than the mocked headless flow; assertions on live metrics and history cards failed.	Adjusted specs to wait for stable rendered states and verify termination at the API boundary.
BUG-2026-07-18-01	Frontend auth guards	All unauthenticated SSR loads redirected to `/login?session_expired=1`, falsely implying an expired session.	Protected routes redirect to `/login`; the `session_expired` flag is reserved for genuine client-side refresh failures.
BUG-2026-07-18-03	`scripts/test-coverage.sh`	`go tool cover` ran from the repo root, so module-relative resolution failed for orchestrator HTML reports.	Coverage renderer now `cd`s into `services/orchestrator` before invoking `go tool cover -html`.
BUG-2026-07-18-12	E2E terminal clipboard	Test targeted xterm's hidden helper textarea instead of the visible terminal surface.	Added `?tab=terminal` deep-link support and updated clipboard spec to open the terminal tab directly.
BUG-2026-07-18-16	Frontend pods page	Launch button was interactable before Svelte attached the click handler, causing flaky E2E confirmation-dialog assertions.	Added a `/pods` hydration marker; E2E helpers wait for hydration before clicking *Launch VM*.
BUG-2026-07-18-18	Landing page navbar	All navbar links pointed to `#features` regardless of label.	Each link now carries a distinct in-page anchor; matching section IDs added.
BUG-2026-07-18-20	Dashboard navigation	No loading skeleton during authenticated route transitions caused visible blank states.	Added `DashboardSkeleton` component; route-specific variants rendered during SvelteKit navigation.

5.4 Sample Test Cases

The six cases below are drawn directly from the repository test suites.

TC-1 — Billing consumer: insufficient-credit grace logic
Suite: `tests/unit/services/test_billing_consumer.py` — *passed* (pytest-junit 2026-07-18). Verifies that when a `billing.deducted` NATS message arrives and the credit deduction raises `InsufficientCredits`, the consumer invokes the grace-period logic rather than nakking or crashing. Test: `test_handle_billing_deducted_runs_grace_logic_on_insufficient_credits`.

TC-2 — Credit service: double-entry balance invariant
Suite: `tests/unit/services/test_credit_service.py` — *passed*. Confirms that `add_credits` produces a transfer record plus exactly two balanced ledger entries that sum to zero, preserving double-entry bookkeeping. Test: `test_add_credits_creates_transfer_and_balanced_entries`.

TC-3 — Orchestrator: pod lifecycle state transitions
Suite: `test-results/contracts/go-contracts.json` — *passed*. The Go contract suite exercises two sub-tests: `TestPodLifecycleValidTransitions` asserts the legal path REQUESTED → RUNNING → TERMINATED, and `TestPodLifecycleRejectsInvalidTransition` confirms that TERMINATED → RUNNING returns an error.

TC-4 — Orchestrator: VM plan billing rates
Suite: `test-results/contracts/go-contracts.json` — *passed*. `TestVMPlanHourlyRates` checks Small=1, Medium=2, Large=4 credits/hour. `TestPartialHourBillingUsesPerMinuteRate` verifies per-minute pro-ration at session end.

TC-5 — E2E: professor credit allocation (TC-CREDIT-004)
Suite: `tests/e2e/specs/credits-and-teaching.spec.ts` — *covered*; included in the final mock-suite run (27 passed, 25.9 s). A Playwright test logs in as a professor, navigates to the credits page, and allocates credits to a student account, then asserts the student's displayed balance increases.

TC-6 — E2E: capacity exhaustion routes to queue (TC-EDGE-005)
Suite: `tests/e2e/specs/pod-lifecycle.spec.ts` — *covered*; final mock-suite run passed. When all capacity is occupied, a launch request returns a queued response and the frontend redirects the user to `/pods/queue`. The test asserts the queue page renders the in-queue entry.

6. CI/CD & Deployment

6.1 Pipeline Overview

Hopper uses **GitHub Actions** for all continuous integration and delivery. Five workflow files live in `.github/workflows/`.

`ci.yml` — Continuous Integration

Triggered on every push or pull-request targeting `main`/`master`. A `concurrency` group cancels in-progress runs on the same ref. The workflow contains **twelve parallel-capable jobs**:

- **frontend** (20 min timeout) — installs pnpm 10 / Node 22, runs `frontend-validate` (TypeScript check + ESLint) and `test-frontend` (Vitest with V8 coverage), stages and uploads coverage artifacts with `if-no-files-found: warn`.
- **api-gateway-smoke** (15 min) — installs Poetry 1.8.4 / Python 3.12 and performs an import smoke test (`from app.main import app`).
- **python-unit** (20 min) — runs the full pytest unit suite (`test-unit`); uploads `coverage/python/uni` and JUnit results.
- **python-migrations** (20 min) — spins up deterministic Docker services (`test-services-up`), runs Alembic migrations (`test-migrate`), captures logs, tears down.
- **python-integration** (30 min) — runs the full pytest integration suite against live containers; uploads XML artifacts.
- **go-orchestrator** (20 min) — runs orchestrator Go tests (`test-orchestrator`), Go 1.23; uploads coverprofile and JSON results.
- **contract-tests** (20 min) — runs both Python and Go contract suites (`test-contract`).
- **helm-chart** (10 min) — runs `helm lint` and `helm template` for `dev`, `staging`, and `prod` value overlays; uses Helm v3.19.0.
- **lint-go** (20 min) — generates proto stubs with `protoc`, then runs `golangci-lint v2.6.2` over both the orchestrator and the contract-test module.
- **e2e-mock** (30 min) — installs Playwright Chromium, starts the deterministic mock stack, runs the full mock-backed E2E suite, captures logs, uploads the Playwright report (retained 14 days).
- **coverage-report** — depends on `frontend`, `python-unit`, `python-integration`, `go-orchestrator`, and `contract-tests`; downloads all coverage artifacts and renders the combined Markdown report.
- **docker** (30 min) — builds all three service images via `scripts/ci/docker-build.sh` as a smoke check (no push).

`publish.yml` — Image Publish & Rollout (CD)

Triggered on push to `main`/`master`, version tags (`v*`), or manual `workflow_dispatch`. Two jobs:

- **build-push** — authenticates to **GHCR** (`ghcr.io`) with `GITHUB_TOKEN`, builds and pushes the three service images (`hopper-api-gateway`, `hopper-orchestrator`, `hopper-frontend`) tagged with the 12-character commit SHA (`ghcr.io/crevios/<image>:<sha12>`).
- **deploy** — conditional on `build-push` succeeding and either a manual `deploy=true` input or the repo variable `AUTO_DEPLOY_K8S=true`. SSHes into the VPS using `VPS_SSH_KEY`/`VPS_HOST`/`VPS_USER` secrets, pipes `scripts/cd/k8s-rollout.sh` over the SSH connection. The rollout script calls `kubectl set image` on `api-gateway`, `orchestrator`, and `frontend` deployments and waits for each rollout with a configurable timeout (default 900 s).

`test-platform.yml` — Extended Platform Tests

Triggered on push to `main`, nightly schedule (`17 2 * * *`), or manual dispatch. Four jobs: `integration-auth-events` (Keycloak + NATS integration suites), `real-stack-e2e` (full real-stack Playwright run with SvelteKit dev server, FastAPI, and Go orchestrator), `load-smoke` (k6 load smoke against the mock stack), and `cross-browser-real-stack-e2e` (opt-in; Chromium + Firefox + WebKit, enabled via `E2E_CROSS_BROWSER=true` or `CROSS_BROWSER_E2E` repo variable).

`staging-validation.yml` — Staging QA

Triggered on manual dispatch or weekly Saturday schedule (`30 3 * * 6`). Runs three parallel jobs against a live staging cluster: `staging-load` (k6 stress/spike/soak profiles against `STAGING_BASE_URL`), `security` (Python security checks with staging KUBECONFIG), and `chaos` (Chaos Mesh YAML manifests from `tests/chaos/`).

`lint-python.yml` runs Ruff on the API Gateway on every push.

ArgoCD (GitOps — partial). An app-of-apps ArgoCD configuration exists at `k8s/argocd/`. The root Application (`hopper-root`) points at `k8s/argocd/apps/`; the child `hopper-platform` deploys `charts/hopper` with `values-prod.yaml`. Sync is **deliberately manual** (no `automated` sync policy, no `prune`) to avoid self-heal reverting CI image rollouts and prune deleting out-of-band Secrets and dynamically-created VM pods. The current live CD path remains `publish.yml`'s SSH-based `kubectl set image`; ArgoCD provides drift visibility and one-click manual sync.

6.2 Environments

Three environments are defined via Helm value overlays in `charts/hopper/` and are verified by the `helm-chart` CI job on every push.

Table 9: Deployment environments

Env	Host / values file	Key differences
Development	`localhost` / `values-dev.yaml`	Keycloak `start-dev`, `secrets.create=true` (in-cluster dev defaults), ingress/TLS/NetworkPolicy/backup all disabled. App services typically run on the host via `docker compose up` + `scripts/deploy-local.sh`.
Staging	`staging.hopper.farefin.com` / `values-staging.yaml`	Single API Gateway replica, Let's Encrypt **staging** issuer (`letsencrypt-staging`, untrusted certs), no nightly backup, secrets provisioned out-of-band.
Production	`hopper.farefin.com` / `values-prod.yaml`	Two API Gateway replicas, Let's Encrypt **prod** issuer, Cilium NetworkPolicies, nightly `pg_dump` CronJob (14-day retention on 10 Gi PVC), SMTP relay via socat on node (port 2525).

Local infrastructure (PostgreSQL/TimescaleDB, NATS JetStream, Keycloak) is managed by `docker-compose.yml`. Image digests are pinned (`timescale/timescaledb:latest-pg16@sha256:0af03...`) to prevent silent major upgrades.

6.3 Deployment Steps

The following procedure is extracted from `README.md`, `scripts/cd/k8s-rollout.sh`, and `k8s/deploy/`.

1. **Build images.** From the repository root:

```
TAG=$(git rev-parse --short HEAD)
REGISTRY=ghcr.io/crevios ./scripts/ci/docker-build.sh
```

   Images produced: `hopper-api-gateway:$TAG`, `hopper-orchestrator:$TAG`, `hopper-frontend:$TA`
   The CI `publish.yml` workflow sets `DOCKER_PUSH=1` to push to GHCR automatically.

2. **Import VM template images** into the k3s containerd runtime (required once per node, not on every deploy):

```
docker save hopper/vm-ubuntu:22.04 | sudo k3s ctr images import -
```

3. **Apply Kubernetes manifests.** Initial cluster setup uses raw manifests in order:

```
kubectl create namespace hopper
kubectl apply -f k8s/deploy/01-infra.yaml    # PostgreSQL, NATS, Keycloak
kubectl apply -f k8s/deploy/02-apps.yaml     # API Gateway, Orchestrator,
    Frontend
kubectl apply -f k8s/deploy/03-ingress.yaml  # Nginx Ingress rules
```

```
kubectl apply -f k8s/deploy/04-network-policies.yaml  # Cilium zero-trust
kubectl apply -f k8s/deploy/06-backup.yaml   # nightly pg_dump CronJob
```

Subsequent deploys use `kubectl set image` via `scripts/cd/k8s-rollout.sh` (invoked by `publish.yml`).

4. **Run database migrations.**

```
docker run --rm --network host \
  -e HOPPER_DATABASE_URL="postgresql+asyncpg://hopper:..." \
  -e PYTHONPATH=/app \
  hopper/api-gateway:latest alembic upgrade head
```

Three Alembic migrations exist: initial schema (001), pod-session column fixes (002), plan/CPU/memory/SSH-port columns (003).

5. **Bootstrap Keycloak.** Create the `hopper` realm, disable SSL for POC deployments, add roles (`admin`, `professor`, `student`), create the initial admin user, and configure redirect URIs for the `hopper-api` public client. Commands use `kcadm.sh` executed inside the Keycloak pod via `kubectl exec`.

6. **Helm-based upgrade** (post-adoption):

```
helm upgrade --install hopper charts/hopper \
  -n hopper -f charts/hopper/values-prod.yaml
```

The chart manages infra, app deployments, ingress, NetworkPolicies, cert-manager ClusterIssuers, and the backup CronJob. Secrets are provisioned out-of-band and must exist before install (`secrets.create=false`).

Open Azure NSG ports (if hosted on Azure): port 443 (HTTPS), port 80 (HTTP), and the NodePort range 30000–32767 (SSH into user VMs) must be open in the Network Security Group.

6.4 Hosting Platform & Live URL

The production cluster is a **k3s** single-node Kubernetes cluster hosted on a **Microsoft Azure** virtual machine. The platform is accessible at `https://hopper.farefin.com`. TLS certificates are issued automatically by **cert-manager** using the Let's Encrypt production issuer (`letsencrypt-prod`), with the TLS secret `hopper-farefin-tls`. The Nginx Ingress Controller routes traffic to the frontend (port 3000), API Gateway (port 8000), and Keycloak (port 8080). Service images are stored in the GitHub Container Registry at `ghcr.io/crevios/` and tagged with the 12-character commit SHA. The current bootstrap image tag recorded in `values-prod.yaml` is `b19bbd699a03`.

6.5 Monitoring & Logging

Observability configuration lives under `observability/`.

Prometheus (`observability/prometheus/prometheus.yml`) scrapes the following targets every 15 s:

- `api-gateway:8000/metrics` and `orchestrator:50051/metrics` (application metrics).
- `nats:8222/metrics` (NATS JetStream consumer lag and message rates).
- `dcgm-exporter` (discovered via Kubernetes endpoint SD) for GPU temperature, VRAM utilisation, and GPU utilisation.
- `node-exporter` (discovered via Kubernetes node SD) for host-level CPU, memory, and disk metrics.

Prometheus is configured with `remote_write` to a **TimescaleDB** instance (the same PostgreSQL pod, via port 9201) for long-term metrics retention.

Alerting. Alert rules in `observability/prometheus/alerts/gpu-alerts.yml` fire to `alertmanager:9093`. Defined rules:

- **GPUTemperatureCritical** — `DCGM_FI_DEV_GPU_TEMP > 90` for 5 min (severity: critical).
- **GPUMemoryNearFull** — VRAM usage above 95% for 2 min (severity: warning).
- **GPUUtilizationStuck** — GPU at 100% for 30 min (severity: warning).
- **DCGMExporterDown** — exporter unreachable for 5 min (severity: critical).
- **PodStuckPending** — pod in `hopper-*` namespace pending for 10 min (severity: warning).
- **NodeUnreachable** — node not Ready for 5 min (severity: critical).
- **NATSJetStreamLag** — consumer pending messages $> 1\,000$ for 5 min (severity: warning).

Grafana dashboard configuration exists at `observability/grafana/dashboards/gpu-overview.jso` (GPU overview). Grafana is not managed by the Helm chart and is deployed separately.

Loki (`observability/loki/loki-config.yml`) is configured with filesystem storage (TSDB schema v13), a 30-day log retention policy, and an in-memory ring KV store. Log ingestion rate is limited to 10 MB/s (burst 20 MB/s).

Structured logging is used throughout the API Gateway via `structlog`. The audit middleware records every mutating HTTP request (method, path, user ID, resource ID) as an immutable audit entry in PostgreSQL.

Health and readiness probes. The API Gateway exposes `/healthz` and `/readyz`. The `/readyz` endpoint checks PostgreSQL connectivity, NATS connectivity, and orchestrator reachability, reporting each dependency individually; an orchestrator outage is reported but does not gate readiness (verified by `test_readyz_orchestrator_outage_is_reported_but_not_g` in the unit suite).

Database backups. A Kubernetes CronJob (`pg-backup`, manifest `k8s/deploy/06-backup.yaml`) runs `pg_dump` nightly at 21:00 UTC using custom format (`-Fc`). Dumps are written to a 10 Gi `local-path` PVC and retained for 14 days.

7. Repository & Documentation Quality

7.1 Branching Strategy & Commit Conventions

Repository. The repository is hosted at `https://github.com/CREVIOS/Hopper`. The default integration branch is **`main`**. At the time of inspection there are 3 local branches (`main`, `Test`, `report`) and 20 remote branches visible via `git branch -a`.

Branch naming. Feature branches follow a `feat/<scope>` pattern (e.g. `feat/HOP-6-web-terminal`, `feat/cpu-vm-admission-queue`, `feat/external-ssh-fix-and-platform-updates`). Fix branches use `fix/<scope>` or plain descriptive names (e.g. `ektu-fix`, `cicd-again`). Documentation branches use `docs/<scope>` (e.g. `docs/vm-queueing`). A `Test` branch is used for iterative CI validation passes.

Merge model. All changes enter `main` via **pull requests**: the log shows 88 commits, the majority carrying a PR number suffix (`#66`–`#115`). GitHub Actions CI gates merges (push/PR trigger on `main`).

Commit conventions. The project uses a subset of **Conventional Commits**. Verified prefixes in the history:

- **`feat`** / **`feat(<scope>)`** — new capability, e.g. *feat(auth): open signup to any email + email verification & password reset (#71).*
- **`fix`** / **`fix(<scope>)`** — bug fix, e.g. *fix(billing): stop ResourceClosedError on idempotent credit deductions (#69).*
- **`docs`** / **`docs(<scope>)`** — documentation only, e.g. *docs(k8s): capture the SMTP relay workaround in the repo (#74).*
- **`chore`** / **`chore(<scope>)`** — maintenance, e.g. *chore: stop tracking scratchpad/ (#73).*
- **`revert`** — undo a previous commit.

Not every commit is prefixed; iterative CI passes use a plain `Test (#N)` message. No `CONTRIBUTING.md` or PR template file was found in `.github/` at the time of inspection.

Code style enforcement. An `.editorconfig` at the repository root enforces: 2-space indent for most files; 4-space for Python; tab indent for Go and Makefiles; LF line endings; UTF-8; trailing-whitespace trimming; final newline. Ruff lints the API Gateway on every push via `lint-python.yml`. ESLint lints the frontend (run inside the `frontend` CI job). `golangci-lint v2.6.2` lints the orchestrator and contract-test modules (`lint-go` CI job). The `.gitignore` excludes `.env`/`.env.*` (with `!.env.example` kept), generated proto stubs, build artefacts, and local IDE directories.

7.2 README Completeness Checklist

The root `README.md` (11 704 bytes) covers the following topics, verified by direct inspection:

Table 10: README completeness audit

Topic	Present
Project description and purpose	✓
Architecture diagram (ASCII art)	✓
Services table (language, port, purpose)	✓
Key flows (VM creation, billing, metrics)	✓
VM plans table (CPU, memory, disk, credits/hour)	✓
Prerequisites list (Docker, kubectl, Go, Node, pnpm)	✓
Quick-start steps (infra, images, migrations, k8s deploy)	✓
Keycloak setup (realm, roles, test user, redirect URIs)	✓
Credit allocation example (curl commands)	✓
Azure NSG port table	✓
Network policies documentation (Cilium, 8 rules)	✓
NATS subject table (publisher, consumer, purpose)	✓
Keycloak OIDC configuration summary	✓
API endpoint reference (auth, pods, credits, admin)	✓
Proto contract paths	✓
Database migration list (3 Alembic versions)	✓
Double-entry ledger design note	✓
SSH-into-VM instructions	✓
Default credentials table (POC)	✓
Environment `.env` variable reference	Partial (`.env.example` only)
Testing instructions	Not present in README
Contributing / PR workflow guide	Not present

A per-service `README.md` exists for the frontend (`frontend/README.md`, 9 571 bytes) covering local dev setup, Vite proxy configuration, dev-login flow, WebSocket origin re-

quirements, and terminal/VS Code connection notes. The `docs/` directory contains `ARCHITECTURE.md`, `SDD.md`, `TESTING_PLAN.md`, `E2E_TESTING.md`, `VM_QUEUEING.md`, `TECH_STACK.md`, `SRS_ADDENDUM.md`, and `TASKS.md`.

7.3 Code Organization / Folder Structure

The repository is organised around service and concern boundaries. The top-level layout (verified by directory listing) is:

Table 11: Top-level repository structure

Path	Contents / Purpose
`services/api-gateway/`	FastAPI application (`app/`), Alembic migrations, Poetry config, Dockerfile, `.env.example`
`services/orchestrator/`	Go module (`cmd/`, `internal/`), Dockerfile, `.env.example`
`frontend/`	SvelteKit application (`src/routes/`, `src/lib/`), Vitest config, Dockerfile, `.env.example`
`proto/`	Protobuf definitions (`hopper/pod/v1/`, `hopper/billing/v1/`) and `buf.yaml`
`k8s/deploy/`	Seven numbered raw YAML manifests (infra, apps, ingress, network policies, cert-manager, backup)
`k8s/argocd/`	ArgoCD app-of-apps root and child Application
`charts/hopper/`	Helm chart with `values.yaml`, `values-dev.yaml`, `values-staging.yaml`, `values-prod.yaml`
`tests/`	All test code: `unit/`, `integration/`, `orchestrator/` (Go), `e2e/` (Playwright), `load/` (k6), `security/`, `chaos/`, `mocks/`, `fixtures/`, `coverage/`
`images/`	VM container image Dockerfiles (`hopper-vm`, `-python`, `-cpp`, `-java`)
`observability/`	Prometheus config + alert rules, Grafana dashboard JSON, Loki config
`infrastructure/`	Ansible playbooks + inventory, Pulumi IaC skeleton
`scripts/`	`ci/` (docker-build), `cd/` (k8s-rollout), `test/` (run.sh), `dev-setup.sh`, `deploy-local.sh`, `generate-proto.sh`, `keycloak-harden.sh`
`docs/`	Architecture, SDD, SRS, testing plan, E2E guide, tech-stack, tasks
`.github/workflows/`	Five GitHub Actions workflow files
`Makefile`	Convenience targets mirroring `scripts/test/run.sh` commands
`docker-compose.yml`	Local infra (PostgreSQL/TimescaleDB, NATS, Keycloak)

Within `services/api-gateway/app/` the internal layout follows FastAPI conventions:

`routers/` (one module per resource group), `services/` (business logic), `schemas/` (Pydantic models), `core/` (database, NATS, auth middleware), `middleware/`, and `proto/` (generated gRPC stubs). Within `services/orchestrator/internal/` the layout mirrors Go package conventions: `k8s/` (pod and network-policy management), `billing/` (ticker), `pod/` (manager), `grpc/` (server), and `config/`.

7.4 Local Setup Instructions

The following steps reproduce the local development environment. They are derived from `README.md`, `scripts/dev-setup.sh`, and the per-service `.env.example` files.

Prerequisites: Docker and Docker Compose, Go 1.23+, Node.js 22+, pnpm 10+ (or 9+), Python 3.12+, Poetry, kubectl with a k3s cluster (optional for orchestrator development).

1. **Clone and configure environment files.**

```
git clone https://github.com/CREVIOS/Hopper.git
cd Hopper
cp .env.example .env
cp services/api-gateway/.env.example services/api-gateway/.env
cp services/orchestrator/.env.example services/orchestrator/.env
cp frontend/.env.example frontend/.env
```

   Edit each `.env` as needed (defaults work for local dev without changes).

2. **Start infrastructure services.**

```
docker compose up -d   # PostgreSQL/TimescaleDB :5433, NATS :4222, Keycloak
    :8080
```

   Wait for PostgreSQL healthcheck: `docker compose ps`.

3. **Install dependencies and run migrations.**

```
# Frontend
cd frontend && pnpm install && cd ..
# API Gateway
cd services/api-gateway && poetry install
poetry run alembic upgrade head && cd ../..
# Orchestrator
cd services/orchestrator && go mod download && cd ../..
```

   Alternatively, `./scripts/dev-setup.sh` performs all of the above steps, including waiting for PostgreSQL readiness.

4. **Start application services** (three terminals):

```
# Terminal 1  Frontend (http://127.0.0.1:5173)
cd frontend && pnpm dev

# Terminal 2  API Gateway (http://127.0.0.1:8000)
cd services/api-gateway
poetry run uvicorn app.main:app --reload --port 8000

# Terminal 3  Orchestrator (gRPC :50051)
cd services/orchestrator && go run ./cmd/orchestrator/
```

5. **Dev login.** Navigate to `http://127.0.0.1:5173` and use *Dev login (skip SSO)* with the credentials from `frontend/.env` (`DEV_LOGIN_USER`/`DEV_LOGIN_PASS`). For a full Keycloak SSO login flow, configure Keycloak as described in the *Deployment Steps* section.

6. **Run the test suite.**

```
make test-unit          # Python unit tests (pytest)
make test-frontend      # Vitest with coverage
make test-orchestrator  # Go orchestrator tests
make test-e2e           # Playwright mock-backed E2E
```

The `make help` target lists all available test commands. All test commands delegate to `./scripts/test/run.sh`.

8. Evaluation & Reflection

8.1 Web Performance & Core Web Vitals

TODO: Add measured Lighthouse or browser-tool performance metrics later.

8.2 Challenges & Solutions

TODO: Add verified implementation or deployment challenges and how they were addressed.

8.3 Limitations & Future Work

TODO: Add realistic limitations and future improvements based on verified project scope.

8.4 Lessons Learned

TODO: Summarize team-level lessons after the full report content is drafted.

8.5 Individual Responsibility

TODO: Replace the placeholder rows with actual team-member responsibilities and contribution evidence.

Member Name	**Core Responsibilities & Feature Ownership**	**Key PRs / Commit Contributions**	**Share (%)**
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %

A. Screenshots / UI Walkthrough

TODO: Add annotated screenshots, UI flows, and captions using image files placed under report/figures/.

Figure 5: Placeholder for a UI walkthrough screenshot loaded from `report/figures/`.